

김용현

System Developer

구조를 설계하고 서비스의 확장성을 만들어가는 개발자

소개

저는 7년차 개발자 김용현입니다.

사내 업무 시스템을 중심으로 신규 프로젝트의 초기 설계와 구조 정의를 주도하며 리드 개발자 및 기술 리딩 역할을 수행해왔습니다. 신규 구축뿐만 아니라 운영 중인 시스템에서도 구조 개선과 방향성 정립을 통해 서비스의 지속적인 확장을 만들어가는 것을 중요하게 생각합니다.

Node.js 기반 개발에 강점을 가지고 있으며, 기술 선택에 있어 팀의 상황과 서비스의 목적에 맞는 방향을 고민하며 적용해왔습니다. 최근에는 React Native 기반 모바일 개발과 AI 기능을 서비스에 접목하는 경험을 확장하고 있습니다.

연락 정보

Email: mse1520@naver.com

기술

Front-End

Javascript Typescript React HTML/CSS

Back-End

Node.js Express TypeORM Elasticsearch

Etc

Git OOP FP Socket.IO

생산 공정 관리 시스템



개요

생산 공장에서 제품 생산에 필요한 자재의 흐름을 관리하고, 공정 데이터를 기반으로 생산 현황 및 통계 정보를 확인할 수 있는 웹 시스템을 구축하였습니다.

공장 내 장비에서 발생하는 데이터를 수집하여 자재 입출고, 비용 관리, 레시피 기반 통계 분석, 실시간 생산 현황 모니터링 기능을 제공하는 것을 목표로 하였습니다.

역할

- 공장 장비 데이터 수집을 위한 데스크탑 애플리케이션 메인 개발
- 데이터 수집 및 서버 전송 구조 설계
- 장비별 통신 프로토콜 분석 및 구현
- 수집된 데이터를 활용한 웹 시스템 기능 개발 참여

기술

WinForms Spring MariaDB TCP/IP

문제

공장 내 자재 입출고 및 생산 데이터가 개별 장비 및 수작업으로 관리되고 있어 데이터가 분산되어 있었고, 이를 통합적으로 관리할 수 있는 시스템이 부재한 상태였습니다.

이로 인해 자재 사용량 및 비용을 정확하게 파악하기 어려웠으며, 실시간 생산 현황 확인이 불가능하여 현장 상황에 대한 대응 및 의사결정이 지연되는 문제가 발생했습니다.

해결

1. 시스템 아키텍처



공장 장비에서 발생하는 데이터는 TCP/IP 통신을 통해 데이터 수집 프로그램으로 전달되며, 수집된 데이터는 로컬 DB에 저장된 후 일정 주기로 API 서버로 전송되어 중앙 DB에 저장됩니다.

이후 웹 시스템을 통해 실시간 모니터링 및 데이터 분석이 가능하도록 구성하였습니다.

2. 데이터 수집 프로그램 구현

공장 내 다양한 장비에서 발생하는 데이터를 통합적으로 수집하고 활용하기 위해, 장비 간 상이한 통신 프로토콜을 유연하게 처리할 수 있는 데이터 수집 구조를 설계하였습니다.

장비별 통신 방식을 분석하여 프로토콜을 설정 기반으로 관리하도록 설계하였으며, 이를 통해 코드 수정 없이 다양한 장비를 유연하게 연동할 수 있도록 하였습니다.

멀티프로세스 기반 구조를 적용하여 여러 장비를 동시에 처리하면서도 장애 격리를 통해 안정성을 확보 하였습니다.

데이터는 로컬 DB에 임시 저장 후 주기적으로 서버 API를 통해 전송하도록 구현하여 네트워크 장애 상황에서도 데이터 유실을 방지하였습니다.

또한 삭제 플래그 기반 데이터 관리 정책을 적용하여 재전송 및 복구가 가능하도록 설계하였습니다.

3. 웹 시스템 구현

실시간 생산 현황 모니터링 화면을 구현하여 장비 상태 및 작업 진행 상황을 직관적으로 확인할 수 있도록 하였습니다.

자재 바코드 또는 사용자 입력을 통해 공정별 자재 투입 정보를 등록하는 레시피 관리 기능을 구현하였습니다.

과거 레시피 데이터를 기반으로 통계 분석 및 추천 기능을 제공하여 공정 효율성을 향상시키고 작업자의 의사결정을 지원하였습니다.

회고

개발자로서 첫 프로젝트였음에도 불구하고 데이터 수집 프로그램의 구조 설계와 구현을 주도적으로 맡으며 큰 책임을 경험한 프로젝트였습니다.

팀 내 관련 경험이 전무한 상황에서 스스로 학습하고 문제를 해결해야 했으며, 이를 통해 문제 정의와 해결 과정의 중요성을 체감할 수 있었습니다.

또한 확장성과 유연성을 고려한 설계의 중요성을 경험하였으며, 이후 다양한 문제 상황에서도 주도적으로 해결할 수 있는 자신감을 얻게 되었습니다.

자재 관리 시스템 재개발



개요

다양한 제품(고압/저압 전동기, 기어박스, 인버터 등)의 자재를 관리하는 기존 시스템이 노후화됨에 따라, 이를 재개발하여 성능 개선과 사용자 경험 향상을 목표로 한 프로젝트입니다.

기존 시스템의 구조적 한계를 개선하고, 기존 사용자들이 익숙한 UI 흐름을 유지하면서도 불편 요소를 개선하여 실사용성을 높이는 데 중점을 두었습니다.

역할

- 프론트엔드 개발 (선임 개발자)
- 주요 업무 화면 설계 및 개발
- 복잡한 화면 구조의 성능 개선 및 구조 개선
- 프론트엔드-백엔드 연동 구조 이해 및 API 연계 처리

기술

Spring Boot jQuery JavaScript SOAP REST API

문제

기존 시스템은 오래된 구조로 개발되어 유지보수성이 낮고, 한 화면에 많은 기능이 결합된 구조로 인해 화면 로딩 속도가 느리고 사용자 경험이 저하되는 문제가 있었습니다.

또한 하나의 페이지에 여러 기능이 밀집되어 있어 코드 간 의존성이 높았고, 이로 인해 기능 추가나 수정 시 영향 범위가 커지는 구조적인 한계가 존재했습니다.

해결

1. 화면 구조 개선 및 성능 최적화

대규모 기능이 하나의 페이지에 집중되어 발생하는 성능 저하 문제를 해결하기 위해, 화면을 기능 단위로 분리하는 구조를 설계하였습니다.

React의 지연 로딩(Lazy Loading) 방식에서 아이디어를 착안하여, 각 기능을 독립적인 모듈 형태로 분리하고 필요 시점에 동적으로 로딩하도록 구현하였습니다. 각 모듈은 개별 URL로 접근 가능한 구조로 개발하고, 스크립트를 통해 필요한 시점에 로딩되도록 구성하여 초기 로딩 속도를 개선하였습니다.

또한 모듈 간 변수 충돌을 방지하기 위해 스코프를 분리하고, 모듈 간 데이터 교환이 필요한 경우에는 별도의 호출 인터페이스를 정의하여 결합도를 낮추고 유지보수성을 향상시켰습니다.

2. 백엔드 연동 구조 처리

백엔드 시스템이 SAP 기반으로 구성되어 SOAP 통신을 사용하는 구조였기 때문에, 프론트엔드에서 직접 사용하기 어려운 형태였습니다.

이를 해결하기 위해 프론트엔드 서버에서 SOAP 통신을 통해 데이터를 수신한 뒤, 이를 REST API 형태로 가공하여 클라이언트에서 호출할 수 있도록 구성하였습니다.

이 과정에서 SOAP 기반 API 구조를 이해하고, 기존 시스템과의 연동을 유지하면서도 프론트엔드에서 활용 가능한 형태로 변환하는 작업을 수행하였습니다.

회고

해당 프로젝트는 레거시 시스템을 개선하는 과정에서 제한된 기술 환경 내에서 최적의 구조를 고민해야 했던 경험이었습니다. React와 같은 최신 프레임워크를 사용할 수 없는 상황에서도, 기존에 알고 있던 개념을 응용하여 유사한 구조를 직접 구현하면서 문제 해결 능력을 키울 수 있었습니다.

특히 하나의 복잡한 화면을 기능 단위로 분리하고 동적으로 로딩하는 구조를 설계하면서, 성능 개선뿐만 아니라 유지보수성과 확장성까지 고려하는 개발의 중요성을 체감할 수 있었습니다.

또한 SOAP 기반 시스템과의 연동을 경험하며 기존 레거시 환경과 현대적인 구조를 연결하는 방식에 대해 이해할 수 있었고, 다양한 기술 스택 간의 차이를 실무적으로 경험할 수 있었던 프로젝트였습니다.

프로젝트 관리 시스템



개요

조직내 인적 자원을 관리하고, 프로젝트별 인력 배치 및 진행 현황을 통합 관리할 수 있는 웹 기반 시스템을 구축하였습니다.

프로젝트별 투입 인력, 기간, 비용, 수익 데이터를 기반으로 프로젝트 상태를 체계적으로 관리하고, 데이터 기반 의사결정을 지원하는 것을 목표로 하였습니다.

역할

- 프로젝트 리드 개발자
- 전체 시스템 아키텍처 설계
- 데이터베이스 테이블 구조 설계
- 프로젝트/인력 관리 기능 설계 및 개발
- 통계 데이터 기반 AI 보고서 자동 생성 기능 기획 및 구현

기술

React React Router Express FastAPI LangChain TypeORM SQLAlchemy

PostgreSQL

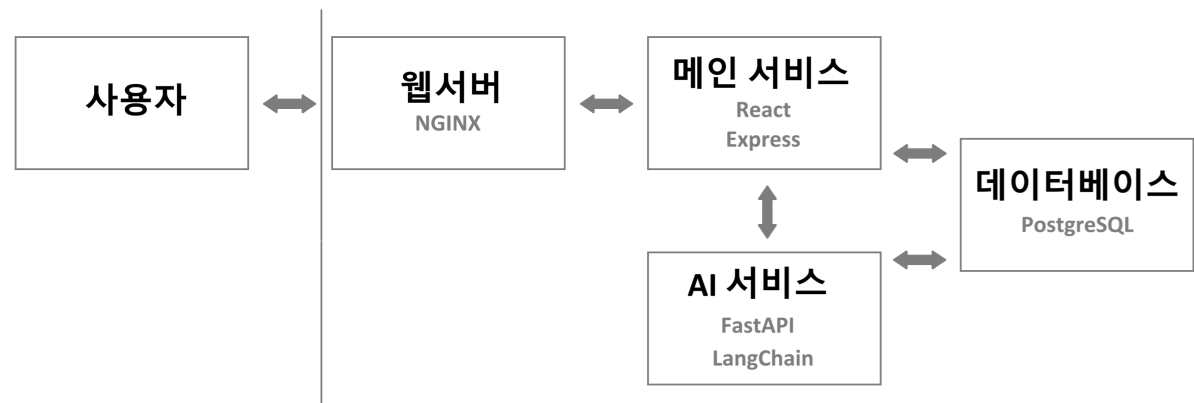
문제

기존에는 프로젝트를 체계적으로 관리하는 시스템이 부재하여 인력 투입 현황과 프로젝트 관련 데이터가 제대로 축적되지 않았고, 이로 인해 리소스 활용 상태와 프로젝트 성과를 파악하기 어려운 구조였습니다.

또한 데이터 기반의 관리 체계가 없어 통계 기반 분석이 불가능했으며, 결과적으로 효율적인 의사결정을 수행하기 어려운 상황이었습니다.

해결

1. 시스템 아키텍처



본 시스템은 메인 서비스(웹)와 AI 서비스로 구성된 구조로 설계하였습니다.

메인 서비스는 사용자 기능 전반을 담당하며, AI 서비스는 보고서 자동 생성 기능을 API 형태로 제공하고 메인 서비스에서 이를 호출하여 사용하는 방식으로 구성하였습니다.

두 서비스는 공통 데이터베이스를 기반으로 동작하며, Docker Compose를 통해 하나의 통합된 서비스로 배포 및 운영되도록 구성하였습니다.

2. 프로젝트 관리 시스템 구현

리드 개발자로서 서비스 전반의 구조를 설계하고, 외부 사용자도 사용할 수 있는 클라우드 서비스 형태를 고려하여 시스템을 구현하였습니다.

기업 단위로 사용 가능한 구조를 설계하여 사용자, 조직(부서), 권한을 유기적으로 관리할 수 있도록 구성하였으며, 인증, 권한, 사용자, 부서 관리 등 핵심 코어 기능을 직접 설계 및 구현하였습니다. 또한 권한 기반 접근 제어와 부서 트리 및 이력 관리 구조를 통해 확장성과 유연성을 확보하였습니다.

프로젝트 관리 기능은 예상 데이터와 실제 데이터를 분리하여 설계하였으며, 역할 기반 인력 구성을 통해 사전 수익성을 분석할 수 있도록 하였습니다. 이후 실제 수행 단계에서는 일일 단위 투입 시간을 기록하여 프로젝트별 비용, 수익, 인력, 기간을 자동 산정하고, 예상 데이터와의 비교 분석이 가능하도록 구현하였습니다.

이를 통해 프로젝트의 비용 흐름과 수익성을 명확하게 파악하고, 데이터 기반 의사결정을 지원하는 시스템을 구축하였습니다.

3. AI 기반 보고서 자동 생성 기능 구현

보고서 작성을 자동화하기 위해, 통계 데이터를 기반으로 AI가 정형화된 보고서를 생성하는 기능을 구현하였습니다.

먼저 보고서의 형식을 표준화하고, 시스템에 저장된 프로젝트 및 인력 관련 통계 데이터를 AI가 활용할 수 있도록 정형 데이터 형태로 가공하였습니다. 이후 해당 데이터를 AI(LLM)에 전달하여, 정의된 보고서 양식에 맞는 문서를 자동으로 생성하도록 구성하였습니다.

생성된 보고서는 미리보기 형태로 제공되며, 사용자는 이를 기반으로 수정하거나 그대로 활용할 수 있도록 구현하였습니다.

회고

이번 프로젝트는 리드 개발자로서 높은 의사결정 권한을 가지고 참여하며, 서비스의 방향성과 기술 구조를 주도적으로 설계한 경험이었습니다. 단순 구현을 넘어 시스템 구조를 스스로 판단하고 결정해야 했기 때문에 개발자로서 한 단계 성장할 수 있는 계기가 되었습니다.

특히 실제 서비스를 설계하는 과정에서 데이터 구조와 권한 체계, 서비스 흐름에 따라 확장성과 유지보수성이 크게 달라진다는 점을 체감하며 설계의 중요성을 깊이 이해하게 되었습니다.

초기에는 프로젝트 관리가 이루어지지 않던 환경에서 관리 체계를 구축하는 것이 목표였으며, 이후 운영 과정에서 사용자에게 더 실질적인 가치를 제공하기 위한 방향을 고민하게 되었습니다.

그 과정에서 AI를 활용한 보고서 자동화 기능을 도입하였고, 반복 업무를 줄이면서 사용자가 체감할 수 있는 편의성을 제공하는 데 집중하였습니다. 해당 기능은 실제 사용자에게 높은 만족도를 얻었으며, 기술 구현이 사용자 경험 개선으로 이어지는 과정을 직접 확인할 수 있었습니다.

이 경험을 통해 사용자 관점에서 문제를 정의하고 해결하는 것의 중요성을 다시 한번 느끼게 되었으며, 서비스의 가치를 높이는 방향으로 고민하는 개발자로 성장할 수 있는 기반이 되었습니다.